

Java Hello World

Roteiro de Atividades

- Nome: Rafael Alves de Oliveira
- Data: 20/08/27

Segue um exemplo de como organizar suas respostas por atividade. Adote o padrão para todas as atividades. Depois de ler isto, apague esta mensagem.

Atividade 1: Explorar IntelliJ

O `==` compara onde estão localizados se estão direcionados para o mesmo lugar, enquanto o `equals` compara diretamente o valor e mesmo sendo números parecidos é algo previsto para otimização, é evidenciado pelos testes que deram `false` no `==` e `true` no `equals`

prompt: Explique o mecanismo de cache de `Integer`, também chamado de "Integer Pool", na especificação da linguagem Java. Explique como funciona o cache de objetos `Integer`, qual é o intervalo garantido pela especificação e por que duas variáveis `Integer` podem ou não apontar para o mesmo objeto quando recebem determinados valores. Não corrija, reescreva ou forneça soluções para nenhum código. Quero apenas uma explicação conceitual do mecanismo.

Resposta: Em Java, `Integer` é uma classe wrapper usada para representar valores do tipo primitivo `int` como objetos. Para melhorar o desempenho e reduzir a quantidade de objetos criados, a JVM utiliza um mecanismo de cache para alguns valores de `Integer`, conhecido como **Integer Cache** ou **Integer Pool**.

A especificação da linguagem Java garante que, para valores inteiros no intervalo de **-128 a 127**, qualquer operação de autoboxing de um valor `int` para `Integer` deve produzir objetos que possam ser comparados por identidade em determinadas situações. Na prática, as implementações da JVM normalmente mantêm em cache os objetos `Integer` desse intervalo.

Por exemplo, quando um valor dentro desse intervalo é convertido para `Integer` por autoboxing, a JVM pode reutilizar um objeto já existente em vez de criar um novo objeto.

Isso é importante porque `Integer` é um objeto, portanto existem duas formas diferentes de comparação:

- `==` compara a **referência**, ou seja, verifica se as duas variáveis apontam para o mesmo objeto.
- `equals()` compara o **valor** armazenado nos objetos.

Assim, dois objetos `Integer` com o mesmo valor podem ser diferentes objetos na memória. Entretanto, para valores que utilizam o cache, é comum que duas variáveis recebam a mesma referência armazenada no pool.

O intervalo de **-128 a 127** é o intervalo mínimo garantido pela especificação para o cache de inteiros. Uma implementação específica da JVM pode utilizar um intervalo maior. Por isso, não é correto depender de `==` para comparar valores de objetos `Integer`, especialmente para valores fora do intervalo garantido.

Em resumo, o Integer Cache existe principalmente como uma otimização para reutilizar objetos `Integer` frequentemente utilizados. Ele não altera o fato de que `Integer` é uma classe e que suas variáveis armazenam referências para objetos. Para comparar o conteúdo de dois objetos `Integer`, o método `equals()` é a forma apropriada.

Sim, a explicação da IA cita o intervalo e a explicação é consistente.

Atividade 2: Escrever o programa Hello World

Sim, eles batem com o valores digitados.

1- coerção 2 - coerção 3- coerção 4- coerção 5- Casting
6- coerção 7 - casting

O casting em java só tira a parte decimal e manter a parte inteira para atribuir o valor prompt:

Explique de forma simples a diferença entre coerção e casting em Java. Por que o casting precisa ser escrito pelo programador no narrowing, enquanto o widening pode ser feito automaticamente? Dê também um exemplo de widening que pode perder um pouco de precisão.

Resposta:

Coerção é quando o Java converte um tipo automaticamente.

Casting é quando o programador força uma conversão escrevendo o tipo desejado.

No **narrowing**, pode haver perda de dados, por isso o Java exige que o programador faça o casting explicitamente.

No **widening**, normalmente não há perda de dados, então o Java faz a conversão automaticamente.

Porém, existe uma exceção: converter `long` para `float` é considerado widening, mas pode perder um pouco de precisão, porque `float` não consegue representar todos os valores de um `long` exatamente.

Exemplo:

```
long x = 123456789;
```

```
float y = x;
```

Nesse caso, o valor pode ser arredondado.